

AP Computer Science Principles

Week 2 (at home): How Computers Work

Today's Objectives:

- Students will be able to explain the fetch-decode-execute cycle and the role of the CPU in executing instructions.
- Students will understand the difference between system software and application software.
- Students will understand the role of computer languages in giving instructions to a computer.
- Students will be able to explain the difference between high-level and low-level computer languages and give examples of each.
- Students will understand the difference between text-based and visual-based programming languages.

Von Neumann Architecture

- The structure and organization of virtually all modern computational devices are based on a single theoretical model called the Von Neumann architecture.
- This architecture is based on the idea that a **central unit** will coordinate a set of instructions for the computer to execute.
- This architecture introduced **memory** as a place to store both instructions and data.
- This architecture streamlined computer interaction into **input and output devices**.
- This architecture follows a sequential process known as the **Fetch-Decode-Execute** cycle.

Computer Hardware

Central Processing Unit (CPU)

- The CPU, often referred to as the "brain" of the computer, performs most of the calculations and instructions in a computer.
- It carries out tasks such as arithmetic operations, logic operations, and controlling the flow of data.
- The CPU consists of the control unit, which manages the execution of instructions, and the arithmetic logic unit (ALU), which performs mathematical and logical operations.



Computer Hardware

Memory (RAM)

- Random Access Memory (RAM) is the temporary storage space where the computer stores data and instructions that are actively being used.
- RAM is **volatile**, meaning its contents are lost when the computer is powered off.
- In the Von Neumann architecture, memory is used to store both data and instructions.
- Memory holds two kinds of contents:
 - Instructions: the program steps the CPU fetches and executes.
 - Data: values the CPU uses during execution, such as variables, inputs, and intermediate results.



Computer Hardware

Storage Devices

- Hard Disk Drive (HDD): HDDs provide long-term storage for programs, data files, and the operating system. They use magnetic disks to store and retrieve data.
- Solid State Drive (SSD): SSDs are similar to HDDs but use flash memory technology for faster data access and improved reliability.
- Optical Drives: Optical drives, such as CD/DVD drives, are used for reading and writing data on optical discs.



Computer Hardware

Motherboard

- The motherboard is the main circuit board that connects and allows communication between various hardware components of the computer.
- It houses the CPU, memory modules, storage devices, and other essential components.



Computer Hardware

Graphics Processing Unit (GPU)

- The GPU, also known as the graphics card, is responsible for rendering and displaying images, videos, and 3D graphics on the computer screen.
- It offloads the graphics-related computations from the CPU, allowing for smooth and efficient visual processing.



Computer Hardware

Input and Output Devices

- Input devices allow the user to input information to the computer: keyboard, mouse, microphone, etc.
- Output devices allow the computer to output information: monitor, printer, speakers, etc.



Fetch-Decode-Execute Cycle

- The Von Neumann architecture follows a sequential process known as the Fetch-Decode-Execute cycle:
 - The CPU **fetches** an instruction from memory
 - **decodes** it to understand the operation to be performed
 - then **executes** the instruction
- This cycle repeats continuously, allowing the computer to process and execute a series of instructions

Computer Software

- **System software** is a collection of computer programs that manage the resources of a computer and facilitate access to those resources.
 - System software acts as an intermediary between the users and the hardware.
 - The set of services and resources created by the software and seen by the user is called a virtual machine.
 - Examples of system software include operating systems (Windows, macOS, Linux), device drivers, and compilers.

Computer Software

- **Application software** allows the user to address some specialized task of interest to that user.
 - They are typically created to meet the needs of individuals, businesses, or specific industries.
 - Examples of application software include word processors, spreadsheets, and web browsers.

Computer Languages

- Once a computer was created, there was a need to communicate with it; but how?
- At their core, computers are a set of **electronic** instructions. These electronic instructions can be broken down to the very basic idea: electricity on and electricity off.
- This is how the computer language of binary came to be (binary has two states: **on and off**)



Computer Languages

- We will discuss more about binary code in Unit 1, but for now, know that the binary language is how a machine is able to execute instruction.
- Binary is a series of 1s and 0s that a computer executes (where 1 represents electricity **on** and 0 represents electricity **off**)
- However, binary is complex and difficult for humans to write and understand. There was a need to develop higher-level languages that are more human-readable and easier to work with.



Pseudocode to High-Level Language

- Unfortunately, there is no way to convert human language into binary directly.
- Human language is riddled with ambiguities and assumptions that computers cannot handle.
- After human language, the next step in the language spectrum is to create a **pseudocode** – an artificial and informal language that helps programmers develop algorithms.
- Pseudocode is a "text-based" design tool.

Human Language

We will input 10 grades into the computer gradebook and print the average of the 10 grades.

Pseudocode

Set sum to zero
Set grade counter to one
While grade counter is less than or equal to ten
 Input the next grade
 Add the grade into the sum
 Increment counter by one
Set the class average to the sum divided by ten
Print the average

- While this provides a great way to write and develop **algorithms**, the computer cannot understand pseudocode.
- After pseudocode, the next step is a programming language.
- A programming language is a formal language with a defined set of rules and **syntax** that allows programmers to write instructions and algorithms for a computer to execute.

<i>Human Language</i>	<i>Pseudocode</i>	<i>Programming Language</i>
We will input 10 grades into the computer gradebook and print the average of the 10 grades.	Set sum to zero Set grade counter to one While grade counter is less than or equal to ten Input the next grade Add the grade into the sum Increment counter by one Set the class average to the sum divided by ten Print the average	sum = 0 counter = 1 while counter <= 10: grade = int(input("Grade:")) sum += grade counter += 1 average = sum / 10 print(average)

Programming Languages

Programming languages provide a structured way for programmers to express their logic, data manipulations, and computational tasks in a format that can be understood by computers. They offer a set of keywords, symbols, and rules that define how programs are written.



- **Syntax**: Programming languages have specific rules and syntax that dictate how programs should be written.
- **Data Types**: Programming languages define different data types that determine how data is stored and manipulated in the program.
- **Variables**: Programmers can declare variables to store and manipulate data during program execution.
- **Control Structures**: Programming languages provide control structures such as conditional statements (if-else, switch) and loops (for, while) to control the flow of execution within a program.

- **Functions and Procedures:** Languages allow the definition and use of functions and procedures, which are reusable blocks of code that perform specific tasks.
- **Libraries:** Programming languages often provide libraries that offer pre-written code for common tasks, allowing developers to use existing coding solutions.
- **Translation:** Programming languages are translated into machine code, either all at once before the program runs (compiled) or line by line as it runs (interpreted).

Examples of popular programming languages include:

- C/C++: Languages known for their efficiency and close interaction with hardware.
- Java: A widely used language known for its platform independence and object-oriented programming features.
- Python: A high-level language known for its simplicity, readability, and extensive libraries.
- JavaScript: A language primarily used for web development, enabling interactivity and dynamic content on web pages.
- Ruby: A language known for its elegant syntax and focus on simplicity and productivity.
- Swift: A language developed by Apple for iOS, macOS, and other Apple platforms.

Compilation Process

Machine Language:

- Machine language is a **low-level** programming language that consists of binary code that can be directly executed by a computer's CPU.
- The machine language is specific to the computer architecture.
- Machine language instructions directly correspond to the operations and data manipulation performed by the computer's hardware.

Assembly Language:

- Assembly language is a **low-level** programming language that provides a more human-readable representation of machine language instructions.
- It uses mnemonic codes or symbols to represent individual machine instructions instead of binary code.
- Each mnemonic corresponds to a specific machine instruction or operation.

Machine Language
0000 0000 0000 0000
0000 0000 0000 0010
0000 0000 0000 0011
0001 1101 0000 0001
0001 1110 0000 0010
0101 1110 1101 1110
Assembly Language
TOTAL .BLOCK 1
ABC .WORD 2
XYZ .WORD 3
LOAD REGD, ABC
LOAD REGE, XYZ
ADD REGF, REGD, REGE
STORE REGF, TOTAL
HALT

Both of these (machine and assembly language) are low-level programming languages – they work closely with the computer, and even though assembly language was a step closer to human language, it was still not great to code in. Then, high level programming languages came around.

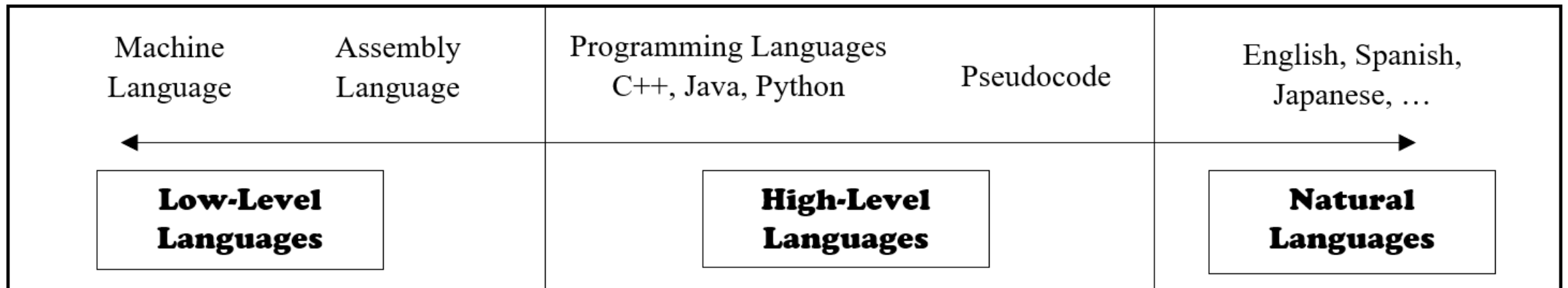
Source Code:

- Source code is a **high-level** programming language, such as Python, Java, or C++.
- It is written in human-readable form, with syntax rules that govern how to give instructions to the computer.
- Source code is eventually translated into machine language by a compiler or an interpreter.

Compilation or Interpretation:

- Depending on the programming language, the source code is either compiled or interpreted.
- In compiled languages like C and C++, a compiler translates the entire source code into machine code, also known as object code, in a process called compilation.
- In interpreted languages like Python or JavaScript, an interpreter reads and executes the source code line by line, converting it into machine code on the fly.

- Low-level languages offer low-level control, direct hardware interaction, and proximity to machine code but lack portability and require detailed knowledge of the hardware.
- High-level languages provide higher-level abstractions, portability, and productivity at the cost of reduced control over hardware resources.



Text vs Visual Programming

- Text-based programming involves writing code using programming languages.
- In this approach, programmers use a text editor or an integrated development environment (**IDE**) to write lines of code using specific syntax and rules.
- Text-based programming allows programmers to have precise control over the logic and behavior of their programs.
- Examples: C++, Java, Python

- Visual-based programming, on the other hand, involves creating programs by manipulating graphical elements or blocks, rather than writing code directly.
- It uses visual programming languages or environments that allow users to build programs by arranging blocks and connecting them together to define the program's flow and behavior.
- Visual-based programming simplifies the coding process by abstracting complex syntax and allowing users to focus more on the program's logic and structure.
- Examples: Scratch, Snap!, App Inventor

Text-Based Programming

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         String var = "thethethe";
5         int count = 0;
6         for(int i = 0; i < var.length() - 2; i++){
7             String word = var.substring(i, i + 3);
8             if(word.equals("the")){
9                 count++;
10            }
11        }
12        System.out.println(count);
13    }
14 }
```

Visual-Based Programming

